

IDOR 漏洞检测智能体演示

使用 AI 智能体自动化识别越权漏洞

概览

- 易受攻击的示例应用：位于 `vulnerable_app/`，涵盖基础版与进阶版两套 Flask 服务
- 检测智能体：位于 `agent/`，提供基础、OpenAPI 驱动与增强型三种检测流程
- 示例脚本：`demo.py` 可一键启动目标应用并串联多个智能体

快速上手

1. 安装依赖

```
pip install -r requirements.txt
```

2. 配置大模型凭证

- **OpenAI 兼容接口**

```
export OPENAI_API_KEY="your-openai-api-key"
export OPENAI_API_BASE="https://api.openai.com/v1" # 如使用官方 API，可省略
export IDOR_MODEL="gpt-4o-mini" # 可选，默认即为 gpt-4o-mini
```

- **DeepSeek 接口**（完全兼容 OpenAI 协议）

```
export DEEPSEEK_API_KEY="你的-deepseek-key"
export DEEPSEEK_API_BASE="https://api.deepseek.com/v1"
export IDOR_MODEL="deepseek-chat"
```

未设置 DeepSeek 变量时，智能体会自动回退到 `OPENAI_API_KEY` / `OPENAI_API_BASE`。

3. 启动易受攻击应用（任选其一）

```
# 基础版：仅包含订单资源
python vulnerable_app/basic_app.py

# 进阶版：包含订单、文档、用户档案并附带 OpenAPI
python vulnerable_app/advanced_app.py
```

4. 在第二个终端运行检测智能体

```
# 基础检测（规则驱动，示例账号 bob）
python agent/simple_agent.py --base-url http://127.0.0.1:5005 --username bob --password bob123

# OpenAPI 驱动检测（自动枚举接口）
python agent/openapi_agent.py --base-url http://127.0.0.1:5005 --username bob --password bob123

# 增强型检测（多资源类型 + 报告生成）
python agent/enhanced_agent.py --base-url http://127.0.0.1:5005 --username bob --password bob123 --verbose
```

5. 查看生成的报告

- `idor_report.md`: 基础/开放 API 智能体输出
- `enhanced_idor_report.md`: 增强型智能体输出

组件说明

易受攻击的应用

- `vulnerable_app/basic_app.py`: 基础订单管理 API, `/orders/<id>` 存在 IDOR 漏洞, `/orders/secure/<id>` 提供加固示例
- `vulnerable_app/advanced_app.py`: 多资源场景（订单 / 文档 / 用户档案），带 `/openapi.json` 供自动化发现

检测智能体

- `agent/simple_agent.py`: 规则驱动探测，自动发现基线资源并生成最小化报告
- `agent/openapi_agent.py`: 解析 OpenAPI，自动枚举对象级路径与安全对照接口
- `agent/enhanced_agent.py`: 多阶段检测、跨资源分析、自动生成专业版 Markdown 报告

辅助脚本

- `demo.py`: 一键启动示例应用并按需运行 `simple / openapi / enhanced / all` 智能体
- `agent/example_usage.py`: 演示如何在代码中以类的方式调用三个智能体

使用示例

- 运行默认演示（启动进阶应用 + 增强型智能体）

```
python demo.py
```

- 针对进阶应用执行 OpenAPI 驱动检测

```
python demo.py --app 2 --agent openapi
```

- 针对基础应用依次运行全部智能体，并使用自定义端口

```
python demo.py --app 1 --agent all --port 5006
```

- 手动测试（按需组合）

```
python vulnerable_app/advanced_app.py # 终端 1
python agent/enhanced_agent.py --username alice --password alice123 #
终端 2
```

示例输出

智能体在运行结束后会输出：

- 访问过的端点、状态码与响应片段
- 是否触发越权访问、涉及的资源 ID 与归属字段
- 自动生成的 PoC（curl 命令）及修复建议
- Markdown 报告文件，便于记录与分享

测试账号

- `alice / alice123` (user_id: 1)
- `bob / bob123` (user_id: 2)
- `admin / admin123` (user_id: 999, 角色: admin)

进阶用法

- 在 `agent/enhanced_agent.py` 中扩展 `build_graph()`，新增自定义检测节点：

```
from agent.enhanced_agent import AgentState, build_graph

def custom_detection_node(state: AgentState):
    # 在此编写自定义逻辑
    return {}

graph = build_graph()
graph.add_node("custom", custom_detection_node)
```